

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 03

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I Jeevan Raaja S (192421005) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	<i>30%</i>	<i>20%</i>	<i>15%</i>	<i>20%</i>	<i>15%</i>	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Jeevan Raaja S

Total Marks :

Annexure A

Short Answer Test

1. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.
2. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.
3. Write a Python program to implement the **Support Vector Machine (SVM)** algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Naïve Bayes Classifier

The question asks you to implement Naïve Bayes, calculate prior and conditional probabilities, classify a test instance, calculate accuracy, and compare results with and without Laplace smoothing.

Aim

To develop a Python program to implement a **Naïve Bayes classifier**, calculate prior and conditional probabilities, classify test instances, evaluate accuracy, and analyze the effect of **Laplace smoothing**.

Concept

Naïve Bayes is a probabilistic classification algorithm based on **Bayes' theorem**. It assumes that the features are conditionally independent given the class.

Bayes' theorem:

Where:

- = probability of class given input
- = probability of input given class
- = prior probability of class
- = probability of the input

For classification, the class having the highest posterior probability is selected.

Algorithm

1. Load the dataset.
2. Separate the features and class labels.
3. Calculate the prior probability of each class.
4. Calculate conditional probabilities for each feature.
5. Calculate the posterior probability for each test instance.
6. Predict the class with the highest probability.
7. Calculate classification accuracy.
8. Apply Laplace smoothing and repeat the classification.
9. Compare the accuracy obtained with and without smoothing.

Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score

data = {
    'Outlook': ['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast',
               'Sunny','Sunny','Rain','Sunny','Overcast','Overcast','Rain'],
    'Temperature': ['Hot','Hot','Hot','Mild','Cool','Cool','Cool',
                   'Mild','Cool','Mild','Mild','Mild','Hot','Mild'],
    'Humidity': ['High','High','High','High','Normal','Normal','Normal',
                'High','Normal','Normal','Normal','High','Normal','High'],
```

```

'Wind': ['Weak','Strong','Weak','Weak','Weak','Strong','Strong',
        'Weak','Weak','Weak','Strong','Strong','Weak','Strong'],
'Play': ['No','No','Yes','Yes','Yes','No','Yes',
        'No','Yes','Yes','Yes','Yes','Yes','No']
}

```

```
df = pd.DataFrame(data)
```

```
X = df.drop('Play', axis=1)
```

```
y = df['Play']
```

```
X_encoded = X.apply(lambda col: pd.factorize(col)[0])
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.3, random_state=42, stratify=y
)

```

```
model_without = CategoricalNB(alpha=0.0)
```

```
model_without.fit(X_train, y_train)
```

```
pred_without = model_without.predict(X_test)
```

```
accuracy_without = accuracy_score(y_test, pred_without)
```

```
model_with = CategoricalNB(alpha=1.0)
```

```
model_with.fit(X_train, y_train)
```

```
pred_with = model_with.predict(X_test)
```

```
accuracy_with = accuracy_score(y_test, pred_with)
```

```
print("Actual Classes      :", list(y_test))
```

```
print("Predicted Without Smoothing:", list(pred_without))
```

```
print("Accuracy Without Laplace Smoothing:",
```

```
      round(accuracy_without * 100, 2), "%")
```

```
print()
```

```
print("Predicted With Smoothing  :", list(pred_with))
```

```
print("Accuracy With Laplace Smoothing:",
```

```
      round(accuracy_with * 100, 2), "%")
```

```

print()

print("Class Prior Probabilities:")
for cls, prob in zip(model_with.classes_, model_with.class_prior_):
    print(cls, ":", round(prob, 4))

test_instance = pd.DataFrame({
    'Outlook': [2],
    'Temperature': [1],
    'Humidity': [0],
    'Wind': [0]
})

prediction = model_with.predict(test_instance)

print()
print("Test Instance Prediction:", prediction[0])

```

Sample Output

Actual Classes : ['Yes', 'No', 'Yes', 'No', 'Yes']
 Predicted Without Smoothing: ['Yes', 'No', 'Yes', 'No', 'Yes']
 Accuracy Without Laplace Smoothing: 100.0 %

Predicted With Smoothing : ['Yes', 'No', 'Yes', 'No', 'Yes']
 Accuracy With Laplace Smoothing: 100.0 %

Class Prior Probabilities:

No : 0.4
 Yes : 0.6

Test Instance Prediction: Yes

The exact test predictions/accuracy can vary depending on the train-test split.

Laplace Smoothing

Laplace smoothing prevents a probability from becoming **zero** when a particular feature value has not appeared in a class during training.

The smoothed probability is:

where is the number of possible feature values.

Comparison

Naïve Bayes	Without Smoothing	With Laplace Smoothing
--------------------	--------------------------	-------------------------------

Naïve Bayes	Without Smoothing	With Laplace Smoothing
--------------------	--------------------------	-------------------------------

Zero probabilities	Possible	Avoided
Unseen feature values	Problematic	Handled
Stability	Lower	Higher
Generalization	May be affected	Usually improved

Analysis

Without Laplace smoothing, an unseen feature-class combination can produce a conditional probability of zero. Since Naïve Bayes multiplies conditional probabilities, one zero probability can make the complete posterior probability zero.

Laplace smoothing adds a small count to every possible feature value. Therefore, it prevents zero probabilities and makes the classifier more robust when unseen combinations occur.

Result

Thus, the **Naïve Bayes classifier** was successfully implemented in Python. Prior and conditional probabilities were used for classification, and the performance was evaluated using accuracy. The comparison shows that **Laplace smoothing helps avoid zero-probability problems and improves the robustness of the classifier.**

Q2. Genetic Algorithm

The question asks you to implement a simple **Genetic Algorithm** to optimize a mathematical function, including population initialization, selection, crossover, mutation, evolution over generations, fitness values, best solution, and parameter analysis.

Aim

To develop a Python program using a **Genetic Algorithm (GA)** to optimize a mathematical function and analyze how parameters such as population size, mutation rate, and number of generations affect convergence speed and solution quality.

Concept

A Genetic Algorithm is an optimization technique inspired by **natural selection and evolution.**

It works with a population of candidate solutions called **chromosomes**. Each chromosome is evaluated using a **fitness function**. The better solutions are selected to produce the next generation through crossover and mutation.

The main operations are:

1. **Initialization** – Create an initial population randomly.
2. **Fitness evaluation** – Calculate the fitness of each solution.
3. **Selection** – Select better individuals as parents.
4. **Crossover** – Combine two parents to create offspring.
5. **Mutation** – Randomly modify some genes.
6. **Replacement** – Form a new population.
7. Repeat the process for several generations.

8. Select the best solution.

Mathematical Function

We will maximize:

where:

The maximum value occurs at:

Therefore:

Python Program

```
import random
```

```
POPULATION_SIZE = 10
```

```
CHROMOSOME_LENGTH = 5
```

```
GENERATIONS = 20
```

```
MUTATION_RATE = 0.1
```

```
def fitness(x):
```

```
    return x ** 2
```

```
def create_chromosome():
```

```
    return [random.randint(0, 1) for _ in range(CHROMOSOME_LENGTH)]
```

```
def chromosome_to_decimal(chromosome):
```

```
    value = 0
```

```
    for bit in chromosome:
```

```
        value = value * 2 + bit
```

```
    return value
```

```
def create_population():
```

```
    return [create_chromosome() for _ in range(POPULATION_SIZE)]
```

```
def selection(population):
```

```
    population = sorted(
```

```
        population,
```

```
        key=lambda c: fitness(chromosome_to_decimal(c)),
```

```
        reverse=True
```

```
    )
```

```
    return population[:2]
```

```
def crossover(parent1, parent2):
```

```
    point = random.randint(1, CHROMOSOME_LENGTH - 1)
```

```

child1 = parent1[:point] + parent2[point:]
child2 = parent2[:point] + parent1[point:]

return child1, child2

def mutation(chromosome):
    for i in range(CHROMOSOME_LENGTH):
        if random.random() < MUTATION_RATE:
            chromosome[i] = 1 - chromosome[i]
    return chromosome

population = create_population()

for generation in range(GENERATIONS):
    best = max(
        population,
        key=lambda c: fitness(chromosome_to_decimal(c))
    )

    best_value = chromosome_to_decimal(best)
    best_fitness = fitness(best_value)

    print(
        "Generation:", generation + 1,
        "Best Solution:", best_value,
        "Fitness:", best_fitness
    )

    parents = selection(population)

    new_population = parents.copy()

    while len(new_population) < POPULATION_SIZE:
        child1, child2 = crossover(parents[0], parents[1])

        child1 = mutation(child1)
        child2 = mutation(child2)

```



```

new_population.append(child1)

if len(new_population) < POPULATION_SIZE:
    new_population.append(child2)

population = new_population

best = max(
    population,
    key=lambda c: fitness(chromosome_to_decimal(c))
)

best_solution = chromosome_to_decimal(best)

```

```

print("\nBest Solution:", best_solution)
print("Best Fitness:", fitness(best_solution))

```

Sample Output

Because the algorithm uses random initialization, the exact output can vary.

Generation: 1 Best Solution: 29 Fitness: 841

Generation: 2 Best Solution: 31 Fitness: 961

Generation: 3 Best Solution: 31 Fitness: 961

Generation: 4 Best Solution: 31 Fitness: 961

Generation: 5 Best Solution: 31 Fitness: 961

...

Generation: 20 Best Solution: 31 Fitness: 961

Best Solution: 31

Best Fitness: 961

Parameter Analysis

Population Size

A larger population provides more candidate solutions and increases diversity. It can improve solution quality but requires more computation.

Mutation Rate

A low mutation rate allows the algorithm to preserve good solutions while introducing some diversity. A very high mutation rate can make the search unstable.

Number of Generations

Increasing the number of generations gives the algorithm more time to improve the solution. However, after reaching a good solution, additional generations may provide little improvement.

Effect on Convergence

Parameter	Low Value	High Value
Population Size	Faster but less diversity	Better diversity, more computation
Mutation Rate	May converge early	More exploration, possibly unstable
Generations	Faster execution	Better chance of finding optimum

Analysis

The Genetic Algorithm gradually improves the fitness of the population through selection, crossover, and mutation. In this problem, the optimal value is **31**, giving a maximum fitness of **961**.

Selection helps preserve good solutions, crossover combines useful characteristics from parents, and mutation introduces diversity. Proper parameter selection is important because a very small population or low mutation rate may cause premature convergence, while excessive mutation may prevent stable convergence.

Result

Thus, the **Genetic Algorithm was successfully implemented** to optimize the mathematical function . The algorithm performed population initialization, selection, crossover, mutation, and evolution over generations. The best solution obtained is $x = 31$, with a fitness value of **961**. The analysis also shows how population size, mutation rate, and number of generations influence convergence speed and solution quality

Q3. Support Vector Machine (SVM)

The question requires a Python implementation of **SVM for binary classification**, using a **linear kernel**, classifying test instances, displaying accuracy, and analyzing the effect of the **margin and support vectors** on classification performance.

Aim

To develop a Python program to implement the **Support Vector Machine (SVM)** algorithm for binary classification using a linear kernel and evaluate its classification performance.

Concept

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification. It finds an optimal **hyperplane** that separates two classes.

For a linear SVM, the decision boundary is:

where:

- w = weight vector
- x = input features
- b = bias

The SVM attempts to maximize the **margin**, which is the distance between the decision boundary and the closest data points from each class.

The closest data points are called **support vectors**.

Important Terms

Margin:

The distance between the separating hyperplane and the nearest training samples. A larger margin generally

provides better generalization.

Support Vectors:

The data points closest to the decision boundary. They determine the position and orientation of the optimal hyperplane.

Linear Kernel:

It is used when the classes can be separated approximately by a straight line or hyperplane.

Algorithm

1. Create a binary classification dataset.
2. Divide the dataset into training and testing data.
3. Create an SVM classifier using a linear kernel.
4. Train the classifier using the training data.
5. Predict the classes of test instances.
6. Calculate the classification accuracy.
7. Identify the support vectors.
8. Analyze the effect of margin and support vectors.

Python Program

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
X = np.array([
    [1, 2],
    [2, 3],
    [2, 1],
    [3, 2],
    [7, 8],
    [8, 9],
    [8, 7],
    [9, 8]
])
```

```
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
model = SVC(kernel='linear', C=1.0)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Actual Classes :", y_test)
```

```
print("Predicted Classes :", y_pred)
```

```
print("Accuracy      :", accuracy * 100, "%")
```

```
print("\nSupport Vectors:")
```

```
print(model.support_vectors_)
```

```
print("\nNumber of Support Vectors:")
```

```
print(model.n_support_)
```

```
print("\nWeight Vector:")
```

```
print(model.coef_)
```

```
print("\nIntercept:")
```

```
print(model.intercept_)
```

Sample Output

Actual Classes : [0 1]

Predicted Classes : [0 1]

Accuracy : 100.0 %

Support Vectors:

[[3 2]

[7 8]]

Number of Support Vectors:

[1 1]

Weight Vector:

[[0.4 0.4]]

Intercept:

[-3.6]

The exact support vectors and output can vary depending on the train-test split and SVM parameters.

Analysis of Margin

The SVM tries to find a decision boundary that maximizes the distance between the two classes.

A **larger margin** generally makes the classifier less sensitive to small variations in the training data and can improve generalization.

If the margin is too small, the classifier may become more sensitive to noise and individual training samples.

Analysis of Support Vectors

Support vectors are the points closest to the decision boundary. They are the most important training points because they determine the position of the separating hyperplane.

If support vectors change significantly, the decision boundary can also change.

For this dataset, the classes are clearly separated, so a linear boundary can classify the test instances accurately.

Effect on Classification Performance

Factor	Effect
Large margin	Generally improves generalization
Small margin	Can make the classifier more sensitive
Important support vectors	Determine the decision boundary
Well-separated classes	Usually gives high accuracy
Overlapping classes	Makes classification more difficult

Result

Thus, the **Support Vector Machine algorithm was successfully implemented** using a linear kernel for binary classification. The model classified the test instances and achieved high accuracy. The analysis shows that the **margin** determines how well the classifier generalizes, while **support vectors** play a key role in determining the optimal decision boundary